能真正落地到现有系统来讲，不只讲概念，而是直接讲：

1. 系统里哪些模块该放进 TEE
2. 远程证明怎么接
3. 密钥怎么发
4. 事件摘要怎么生成
5. 差分隐私具体加在哪些统计接口上
6. 七天内怎么做成“可答辩版”
7. 后续怎么升级成“较完整版”

先给一个总判断：

现在要做“可实现、可答辩、可写进作品报告”的版本，最合理的路线是：

- **主线**：TEE 保护“视频处理中的数据”
- **辅线**：差分隐私保护“平台对外展示的聚合统计”

这和机密计算的基本定位是一致的。CCC 将 confidential computing 定义为用基于硬件的 TEE 保护“使用中的数据”；Azure Attestation 这类服务则专门做远程证明，验证运行环境与二进制是否可信，并可据此发放令牌或释放密钥。NIST 则把差分隐私视为目前最严格的数学隐私定义之一，尤其适合统计发布场景。

一、这套系统里，TEE 和差分隐私各自负责什么

1. TEE 负责保护“原始视频”和“处理中间结果”

最敏感的数据不是数据库里的表，而是：

- 摄像头原始帧
- 解码后的图像
- 人脸位置
- 人体框、轨迹、位移
- 风险判定前的中间特征
- 可能用于复核的匿名化片段

这些都属于 data in use，也就是“正在被算法处理的数据”。TEE 的价值就在这里。CCC 明确强调 TEE 用来保护使用中的数据；Azure Attestation 的远程证明机制，则是让外部确认“确实是指定程序运行在可信平台上”，再决定是否发令牌、放行请求或释放密钥。

2. 差分隐私负责保护“统计结果”

差分隐私不要加在实时识别主链路上，不然会影响识别效果。
它应该只加在这些**聚合统计接口**上：

- 某日跌倒总次数
- 某周高风险事件数
- 夜间异常分布
- 不同时间段告警趋势
- 按房间/按用户的统计图表

NIST 2025 的指南也把差分隐私和“统计发布”紧密联系在一起。
(nvlpubs.nist.gov)

二、现有系统的改造目标

当前系统大致是：

摄像头/视频
→ 后端处理
→ 风险引擎
→ 事件记录
→ 前端展示

改造后变成：

摄像头/视频
→ TEE 内部：解码
→ TEE 内部：匿名化
→ TEE 内部：检测与追踪
→ TEE 内部：风险判定与事件摘要生成
→ TEE 外部：事件数据库
→ 统计接口加差分隐私
→ 前端展示

核心原则是：

- **原始帧不离开 TEE**
- **TEE 外只接收事件级结果**
- **统计结果默认做轻量扰动**
- **任何需要查看图像的功能，只允许看模糊后短片段或快照**

三、TEE 方案怎么具体实施

这里给一个最适合写进作品报告的技术实现版本：

方案名称

基于可信执行环境的边缘侧隐私增强计算模块

采用方式

建议在报告中写：

本系统采用 TEE 兼容的机密计算部署思路。核心敏感处理链路以 enclave/可信容器方式封装，普通业务逻辑与管理接口运行在非可信区。通过远程证明建立边缘端可信身份，只有在证明通过后，系统才允许密钥注入、配置下发和事件上报。

这个表述非常稳。

四、TEE 内外模块边界怎么划分

这一步很重要。一定要让评委看到：
不是“整个系统都塞进 TEE”，而是“只把最敏感、最必要的部分放进去”。

放进 TEE 的模块

这些模块直接处理原始视频或可还原隐私的中间结果，应放进 TEE：

1. 视频帧接收与解码模块

职责：

- 接收摄像头流或视频文件帧
- JPEG/H.264 解码
- 形成内存中的原始图像张量

原因：
解码后的图像是最敏感数据之一。

2. 匿名化模块

职责：

- 人脸检测
- 人脸模糊
- 身体区域裁剪
- 轮廓化/前景分割/骨架化
- 可选：直接丢弃背景

原因：
匿名化前仍是高敏图像，必须在可信区完成。

3. 检测与追踪模块

职责：

- YOLO 推理
- ByteTrack 追踪
- 位移、速度、长宽比变化计算
- 静止时长与夜间活动特征提取

原因：
这些特征虽然比原始视频低敏，但仍可反推个人行为模式。

4. 风险判定模块

职责：

- 跌倒连续帧判断
- 静止滑动窗口判断
- 夜间异常检测
- 风险升级逻辑

原因：
这里把原始特征转成高层事件，是“脱敏”的关键关口。

5. 事件摘要生成模块

职责：

- 输出事件 JSON
- 仅保留必要字段

- 对 person_id 做匿名映射
- 生成必要的模糊后截图

原因：
这是 TEE 与外部世界的数据边界。

放在 TEE 外的模块

这些不直接接触原始视频，可以放在普通服务里：

- 用户登录认证
- Dashboard 图表
- 事件列表分页
- 系统配置页面
- 告警推送模块
- 权限管理
- 统计分析接口
- 差分隐私扰动服务

五、TEE 内外通信怎么设计

需要明确两类接口：

1. TEE 输入接口

只允许输入这几类内容：

- 摄像头帧数据
- 模型文件哈希/版本号
- 风险阈值配置
- 时间配置（夜间时段）
- 密钥材料或令牌

2. TEE 输出接口

只允许输出这几类内容：

- event_type

- risk_level
- timestamp
- duration_sec
- anon_person_id
- feature_summary
- blurred_snapshot_ref
- attestation_status
- model_version

推荐的事件摘要结构

```
{
  "event_id": "evt_20260413_00128",
  "event_type": "fall",
  "risk_level": "HIGH",
  "timestamp_start": "2026-04-13T09:12:21+08:00",
  "timestamp_end": "2026-04-13T09:12:29+08:00",
  "duration_sec": 8.0,
  "anon_person_id": "p03",
  "feature_summary": {
    "center_drop_ratio": 0.68,
    "aspect_ratio_change": 1.73,
    "motion_energy_mean": 0.05,
    "stillness_window_sec": 6.2
  },
  "snapshot_type": "blurred",
  "snapshot_path": "/secure/event_snaps/evt_20260413_00128.jpg",
  "model_version": "fall_v1.2.3",
  "tee_measurement": "mr_enclave_or_measurement_hash"
}
```

六、远程证明怎么接进来

这一步是 TEE 从“隔离执行”升级成“可信系统”的关键。

Azure Attestation 这类服务的核心作用是：接收 enclave/TEE 证据，按策略验证，再签发可供依赖方使用的证明令牌；Open Enclave 也正是为了给不同 TEE 提供统一开发抽象。Microsoft 的文档明确写到 Azure Attestation 可验证 enclave 证据、根据客户策略评估，并为依赖方生成签名令牌；Open Enclave 则是一个跨平台统一 enclave 抽象。(Microsoft Learn)

具体流程

步骤 1：TEE 启动

- 边缘端设备启动 enclave/可信容器
- 加载固定版本的匿名化和检测代码
- 生成 measurement（代码度量值）

步骤 2：生成 attestation evidence

- enclave 生成 report / quote
- 对于 SGX 路线，Intel 的 DCAP 提供 quote generation 相关能力；Intel 文档说明了 ECDSA quote generation library 用于生成基于 ECDSA 的远程证明 quote。(Intel)

步骤 3：提交给 attestation 服务

- 发送到 Azure Attestation 或自建验证服务
- 服务验证：
  - 平台是否可信
  - enclave measurement 是否匹配
  - 策略是否允许
  - TCB 是否通过

步骤 4：获取 attestation token

- 若通过，返回带签名的 attestation token
- token 内可包含 measurement、时间、策略结论等 claims

步骤 5：用 token 换取密钥

- 只有 attestation 通过，才允许：
  - 获取模型解密密钥
  - 获取数据库写入令牌
  - 获取事件上报密钥
  - 获取模糊截图上传密钥

这一步特别好讲，因为 Azure Attestation 官方就明确提到：证明通过后，可用于向 Key Vault / HSM 等系统请求密钥释放。(Microsoft Learn)

七、密钥管理怎么做

这个部分非常容易在答辩里加分，因为它体现“不是只隔离运行，还把权限控制做进去了”。

建议分三类密钥

1. 模型密钥

用途：

- 解密 ONNX 模型文件
- 防止模型被随意复制或篡改

流程：

- 模型文件磁盘上加密存储
- enclave 启动后通过 attestation token 请求解密密钥
- TEE 内部完成解密并加载模型
- 明文模型不落盘

2. 事件上报密钥

用途：

- 给事件摘要做签名
- 保证事件来自可信 TEE，不是伪造的

流程：

- enclave 内持有私钥或临时会话密钥
- 每条事件摘要带签名
- 服务端验签后才入库

3. 敏感快照访问密钥

用途：

- 控制模糊后截图或短片段的访问
- 只有有权限的用户才能查看

流程：

- TEE 输出模糊后图像
- 图像对象存储加密
- 访问时通过权限系统签发短时 URL

八、现有 Python 系统怎么落地

后端是 Python 为主，所以这里给一个务实路线。

路线 A：比赛版，逻辑隔离 + TEE 兼容设计

做法

- 把核心处理链封装成独立服务 secure_core
- 对外只暴露两个接口：
  - process_frame()
  - close_session()
- 所有原始帧只发给 secure_core
- 普通 Quart 后端不接触原始帧，只接 event summary

代码结构建议

```
backend/
  app.py
  api/
  services/
  secure_core/            # 未来可部署到 TEE
    enclave_api.py
    anonymizer.py
    detector.py
    tracker.py
    risk_engine.py
    event_builder.py
  storage/
  dp_stats/
```

作用

即使短期内没有真实 TEE 硬件，也已经完成了“可信边界”的软件分层。
答辩时可以说：

当前原型已按 TEE 兼容方式拆分为可信计算核心与普通业务服务两部分，后续可直接映射到 Open Enclave / Confidential Container 部署。

这个说法合理。

路线 B：较完整版，Open Enclave / Confidential Container

Open Enclave 官方定位就是提供统一 enclave 抽象，适合把“host + enclave”这种结构做出来。(Open Enclave)

实施步骤

1. 用 C/C++ 或支持的桥接方式重写最小 enclave 核心
   只保留：
   - 人脸模糊
   - 检测结果后处理
   - 风险判定
   - 事件摘要构建
2. Host 进程负责：
   - 摄像头输入
   - 文件 I/O
   - 网络通信
   - 数据库存储
3. Enclave 负责：
   - 敏感数据内存处理
   - 关键特征计算
   - 事件签名
4. 用 Azure Attestation 做证明
   Azure 文档明确提供了 Open Enclave attestation 的 REST API 路径。(Microsoft Learn)

适合怎么写

可以写成：

- 原型：逻辑等价实现
- 工程增强：可迁移至 Open Enclave / 机密容器部署

九、差分隐私具体怎么加

这一部分不要空泛，要非常明确。

1. 只对“统计查询接口”加，不对实时识别加

加 DP 的接口包括：

- /api/stats/daily
- /api/stats/weekly
- /api/stats/risk_distribution
- /api/stats/night_events
- /api/stats/fall_trend

不要对这些加：

- 实时检测结果
- 单条事件详情
- 风险告警即时推送

2. 选择保护对象：用户级或事件级

NIST 特别强调“privacy unit”必须定义清楚，不同定义会显著影响保证强度。(nvlpubs.nist.gov)

最适合定义为：

方案一：用户级隐私

一个老人/一个房间/一个家庭是一个 privacy unit。
这是最稳的答辩说法。

方案二：事件级隐私

一条事件是一个 privacy unit。
更容易实现，但隐私保证更弱。

建议报告中写：

本系统统计发布默认采用用户级隐私单元，以控制聚合图表对个体行为模式的反推风险。

---

3. 选择机制

统计图大多数是计数查询，所以最适合：

Laplace 机制

用于：

- 总次数
- 分类计数
- 柱状图、饼图

公式层面不用展开太多，可以只写：

- 真实值 c
- 输出 c + Lap(Δ/ε)

其中：

- Δ 是敏感度
- 对简单计数查询通常取 1
- ε 控制隐私与准确度权衡

4. 参数建议

比赛里不要把参数写得太激进。

建议：

- 单个统计接口：ε = 0.5 ~ 1.0
- 每日看板总预算：ε_total = 2 ~ 3
- δ = 0 或不写，直接采用纯 DP 的 Laplace 机制

因为是“轻量差分隐私”，不是全国人口普查级系统，不需要把预算体系写得太复杂。

十、差分隐私统计模块怎么实现

模块位置

```
backend/
  dp_stats/
    budget_manager.py
    mechanisms.py
    postprocess.py
    stats_service.py
```

各模块职责

budget_manager.py

负责：

- 记录每个接口用了多少 ε
- 控制一天内的总预算
- 超预算则降级展示

mechanisms.py

负责：

- 拉普拉斯噪声生成
- 整数裁剪
- 非负约束

postprocess.py

负责：

- 小于 0 的计数截断为 0
- 四舍五入
- 保证图表可读性

NIST 也特别提醒：后处理不影响隐私保证，但会影响偏差和实用性，比如把负数截断为 0 会引入统计偏差。(nvlpubs.nist.gov)

stats_service.py

负责：

- 查询数据库原始计数
- 调用 DP 机制
- 输出给前端图表

代码思路示例

```
import numpy as np

def dp_count(true_count: int, epsilon: float, sensitivity: float =
1.0) -> int:
    noise = np.random.laplace(loc=0.0, scale=sensitivity / epsilon)
    noisy = true_count + noise
```

```
    return max(0, int(round(noisy)))
```

使用示例

- 原始跌倒次数：12
- ε = 0.8
- 噪声后输出：11 或 13 或 10

这很适合统计图，但不适合实时告警。

十一、如何避免差分隐私把图表搞得太离谱

这里有几个实操建议：

1. 低频统计不显示精确数

比如某天只有 1 次事件，噪声后波动会很明显。
做法：

- 少于阈值时显示“低频”或“<5”
- 不显示具体值

2. 合并时间粒度

不要按分钟统计。
按天、按周展示更稳。

3. 限制重复查询

同一个图表如果反复刷新，会消耗预算。
做法：

- 当天同一用户同一图表固定返回同一扰动结果
- 结果缓存 10 分钟或 1 小时

4. 不对单用户事件详情做 DP

事件详情本身不是“统计发布”，加噪声会让系统不可用。

十二、给一个完整的端到端执行流程

下面这个流程最适合直接写进技术方案。

阶段 1：可信启动

1. 边缘端启动 secure_core
2. 加载 enclave/可信容器
3. 生成 measurement
4. 发起 attestation
5. attestation 服务验证成功
6. 返回 token
7. 用 token 从密钥服务获取模型密钥与事件签名密钥

阶段 2：视频进入 TEE

1. 摄像头采集一帧图像
2. 帧送入 TEE 内部缓冲
3. TEE 内完成人脸模糊与背景降敏
4. TEE 内执行检测与追踪
5. TEE 内计算轨迹、位移、静止时长、姿态变化
6. TEE 内完成跌倒/静止/夜间异常判定
7. TEE 内构造事件摘要
8. TEE 对事件摘要签名后发给后端

阶段 3：服务端处理

1. 后端验签
2. 写入事件数据库
3. 若风险高，则触发告警
4. 保存模糊后截图引用
5. 前端展示事件卡片

阶段 4：统计展示

1. 用户请求 Dashboard
2. 后端先查询真实计数
3. 调用 DP 机制加噪

4. 扣减隐私预算
5. 输出扰动后的图表数据

十三、七天内怎么做成“可答辩版”

这是最关键的。

第 1 天：重构可信边界

- 把现有检测链封装成 secure_core
- 保证原始帧只进 secure_core
- 其他模块只收 event summary

第 2 天：加“事件级输出”

- 删除原系统里不必要的原始视频传递
- 明确输出字段
- person_id 改匿名 ID
- 默认不存原始视频

第 3 天：加“伪 attestation 流程”

如果真 TEE 来不及，就做逻辑等价版：

- 启动时输出 measurement hash
- 后端校验白名单 hash
- 校验通过才允许服务启动
- 演示中把它命名为“原型级远程证明流程”

注意：答辩时别说已经实现完整硬件级 attestation。
要说：

当前原型实现了 attestation-compatible 流程，正式部署可接入 Open Enclave / Azure Attestation。

第 4 天：加密钥放行逻辑

- 模型文件加密存储
- 服务启动后校验 measurement

- 校验通过才解密模型

这一步非常加分。

第 5 天：加差分隐私统计

- 只对 Dashboard 图表接口加 DP
- 做预算管理
- 做负值截断
- 做缓存

第 6 天：补文档与图

- 一张 TEE 数据流图
- 一张 attestation + key release 图
- 一张 DP 统计输出图

第 7 天：答辩打磨

重点练三句话：

1. 们保护的是“使用中的视频数据”
2. 原始视频不进入平台主数据流
3. 对外统计采用轻量差分隐私，避免从图表反推个体行为模式

十四、作品报告里可以怎么写

这段可以直接作为正式文字骨架：

本系统在隐私计算层面采用“机密计算 + 差分隐私”的双层保护方案。
在输入侧和计算侧，系统基于可信执行环境兼容的部署思路，将视频帧解码、人脸匿名化、人体检测、多目标追踪、风险判定和事件摘要生成等核心敏感流程封装于可信计算核心中执行。系统通过远程证明验证边缘端运行环境与核心二进制的一致性，仅在证明通过后释放模型解密密钥与事件签名密钥，从而保证原始视频及其中间特征在处理过程中不暴露于普通业务进程。
在输出侧和展示侧，系统对统计分析模块引入轻量差分隐私机制，仅对日/周事件计数、风险分布、夜间异常趋势等聚合统计结果进行扰动发布，而不影响实时检测与单条事件处置。该设计在保证系统可用性的同时，降低了长期聚合结果泄露个体行为模式的风险。

因此，本系统的隐私保护不再局限于“人脸模糊”或“减少存储”，而是扩展为覆盖数据采集、使用、上报和统计发布全过程的隐私增强计算框架。

十五、一个务实提醒

一定要区分这三种表达：

可以说

- TEE 兼容部署
- 原型实现了可信边界划分
- 已实现 attestation-compatible 启动校验流程
- 正式部署可接入 Open Enclave / Azure Attestation
- 已实现差分隐私统计输出模块

不建议直接说

- 已完整实现硬件级远程证明
- 已完成生产级机密计算部署
- 已达到严格密码学安全证明

这样更稳。

十六、对最推荐的最终落地形态

比赛版最优组合：

- 软件层完成 secure_core 拆分
- 原始帧不进入普通后端
- 事件摘要签名
- 模型文件加密 + 启动校验放行
- Dashboard 统计加拉普拉斯噪声
- 文档中写成“TEE 兼容 + 可接 attestation”

这个版本最符合现在的时间条件。

升级版形态：

- Open Enclave / Confidential Container
- Azure Attestation 接真实远程证明
- attestation 成功后从 Key Vault 放密钥
- 统计层加入预算管理和缓存策略

这个版本适合写成“后续增强方案”。

TEE + 轻量差分隐私 成一套能理解、能答辩、能改进现有系统的方案。

可以先把它理解成一句话：

TEE 负责把“正在处理的视频”关进一个安全小房间里算；差分隐私负责把“最后展示给别人看的统计图表”稍微加一点模糊，让别人看趋势，但不容易反推出某个老人具体发生了什么。
TEE/机密计算的核心目标是保护“使用中的数据”，也就是数据正在被处理时的安全；差分隐私则是一个量化“统计结果泄露风险”的数学框架。
(Confidential Computing Consortium)

一、先把两个概念讲明白

1. 什么是 TEE

TEE，中文常叫**可信执行环境**。
可以把普通电脑想成一栋大楼，里面很多程序都在一起办公。正常情况下，视频、检测结果、模型文件、临时特征这些东西都在这栋大楼里跑。问题是，只要系统被入侵、程序有漏洞、管理员权限过大，**这些“正在被处理”的数据就有机会被看到**。

TEE 就像在大楼里再修一间**带保险门的机密房间**：

- 房间外的人知道里面在运行程序
- 但房间外不能随便看到里面的数据
- 也不能随便篡改里面的程序
- 甚至外部还可以先检查“房间里运行的是不是信任的程序”，这一步就叫**远程证明（attestation）**

Confidential Computing Consortium 对机密计算的核心描述就是：借助硬件支持的 TEE 来保护“使用中的数据”。Azure Attestation 的官方说明也明确说，它可以远程验证平台是否可信、里面运行的二进制是否完整，并生成带签名的证明令牌。(Confidential Computing Consortium)

---

2. 什么是差分隐私

差分隐私可以理解成：
当系统发布一个统计结果时，别人很难仅仅因为“某个人的数据在不在里面”就推断出那个人的隐私。

它不是“把原始数据加密”，也不是“把图像打码”，而是：

- 对**统计结果**做处理
- 加入一点经过设计的随机噪声
- 让整体趋势还能看
- 但单个个体的影响更难被反推出去

NIST 2025 的指南把差分隐私定义为一种数学框架，用来量化当某个实体的数据出现在数据集中时，系统会产生多大的隐私损失。(NIST Computer Security Resource Center)

二、为什么这个项目适合“TEE + 轻量差分隐私”

因为的项目有两个很明显的隐私风险点。

1. 第一类风险：原始视频和处理中间结果太敏感

独居老人居家视频不仅有人的动作，还会暴露：

- 房间布局
- 起居时间
- 身体状态
- 是否独居
- 夜间活动规律
- 有无摔倒、起身困难等情况

这些最敏感的数据，恰恰是在**解码、检测、追踪、风险判定**时被系统看到的。所以这里最适合用 TEE：保护“处理中的数据”。(Confidential Computing Consortium)

2. 第二类风险：长期统计图表也可能泄露模式

即使不上传原始视频，只展示统计图，比如：

- 本周夜间异常 8 次
- 某房间近 30 天跌倒 3 次
- 最近 7 天高风险告警集中在凌晨 2 点到 4 点

这些聚合图表如果长期累积，也可能被别人推断出这个老人最近身体变差、夜间活动异常、某段时间经常起夜。
所以统计发布层适合加**轻量差分隐私**。(NIST Computer Security Resource Center)

三、最通俗的整体架构图应该怎么理解

可以给学生讲成下面这条链路：

摄像头
→ TEE 小房间里解码视频
→ TEE 小房间里做人脸模糊、前景提取
→ TEE 小房间里做人体检测、追踪、跌倒判断
→ TEE 小房间里只生成事件摘要
→ 外面的普通系统只拿到“事件结果”
→ 做图表时再加一点差分隐私噪声

也就是说：

- **TEE 解决“原始视频怎么安全地被处理”**
- **差分隐私解决“统计结果怎么安全地被展示”**

四、先讲 TEE：学生最容易懂的版本

1. TEE 到底保护什么

TEE 不是保护所有东西，它主要保护这三类内容：

第一类：原始帧

比如摄像头拍到的一张图片。
这是最敏感的。

第二类：中间特征

比如：

- 人脸位置
- 身体框
- 轨迹
- 位移
- 倾斜角度
- 静止时间

这些虽然没有原始视频那么敏感，但仍可能反推出个人行为习惯。

第三类：关键程序和模型

比如：

- 的人脸模糊模块
- YOLO 模型
- ByteTrack 模块
- 风险判定规则
- 事件摘要生成器

TEE 不只是保护数据，也要保护“里面运行的程序确实是想要的那份程序”。Azure Attestation 的官方文档就强调，会验证平台可信性和二进制完整性。(Microsoft Learn)

---

2. TEE 不保护什么

这点也要跟学生说清楚，不然容易神化。

TEE **不是万能保险箱**。它通常不负责：

- 前端页面长什么样
- 数据库表怎么设计
- 用户密码是否太弱
- 管理员有没有乱授权
- 网络接口有没有逻辑漏洞

所以 TEE 是**隐私保护的一部分**，不是整个系统安全的全部。这个表述也比较符合 CCC 对机密计算的定位：重点是“保护使用中的数据”。(Confidential Computing Consortium)

五、系统里哪些模块该放进 TEE

这部分很关键，学生一看就能明白“到底搬什么进去”。

1. 放进 TEE 的模块

模块 A：视频接收与解码

作用是把摄像头流或者视频文件变成一帧一帧图像。

为什么要放进去？
因为图像一旦解码出来，就是完整原始画面，最敏感。

模块 B：匿名化模块

作用包括：

- 人脸检测
- 人脸模糊
- 背景弱化
- 前景提取
- 可选的轮廓化、骨架化

为什么要放进去？
因为匿名化之前的数据还是完整隐私数据，必须在可信环境里完成。

模块 C：检测与追踪模块

作用包括：

- YOLO 检测人或跌倒
- ByteTrack 给每个人分配轨迹 ID
- 计算位移、速度、长宽比变化

为什么要放进去？
因为这些特征能反推个人行为模式。

模块 D：风险判定模块

作用包括：

- 连续几帧判断跌倒
- 静止超过多少秒算异常
- 夜间是否异常停滞
- 风险从 LOW 升到 MEDIUM 或 HIGH

为什么要放进去？
因为这里是“从低层特征变成高层事件”的关键位置。

模块 E：事件摘要生成模块

作用包括：

- 把原始复杂信息压缩成结果
- 输出事件类型、时间、风险等级、持续时长
- 生成匿名 person_id
- 生成模糊后的截图引用

为什么要放进去？
因为这是 TEE 和外部世界的边界。

---

2. 不需要放进 TEE 的模块

普通后端模块

比如：

- 用户登录
- 页面查询
- 事件列表分页
- 可视化图表
- 系统配置页面
- 告警通知
- 日志管理

这些只需要接收**事件摘要**，不需要看原始视频。

六、远程证明（attestation）到底是什么

这是学生最容易懵的一部分，用最简单的方式解释。

1. 为什么还要远程证明

就算说“这里有 TEE”，别人也会问：

怎么证明这个 TEE 里运行的真的是说的那段代码？

这时就要用**远程证明**。

它的本质是：

- TEE 自己生成一份“身份证明”
- 外部验证机构检查这份证明
- 如果通过，就发一个“可信令牌”
- 只有拿到这个令牌，系统才给它密钥或允许它上报数据

Azure Attestation 官方就明确把这件事描述为：接收证据、按策略验证、生成带签名的证明令牌，供依赖方信任使用。(Microsoft Learn)

---

2. 远程证明可以类比成什么

可以对学生说：

远程证明就像考试进考场前的“人脸识别 + 准考证检查 + 金属探测门”：

- 先看是不是本人
- 再看证件是不是真的
- 再看有没有带不该带的东西
- 全都通过了，才放进考场

TEE 的远程证明也类似：

- 看平台是不是可信硬件
- 看里面的程序是不是指定版本
- 看有没有被篡改
- 都通过了，才发密钥

七、TEE 里的完整执行流程，按步骤讲

下面这部分可以直接给学生当“操作流程说明”。

第一步：TEE 启动

边缘设备启动时，先加载可信计算核心。
这个核心可以理解成“机密小程序”。

它启动后会生成一个**度量值**，也可以简单理解成“程序指纹”或“程序哈希”。

这个指纹能帮助外界判断：
“里面现在运行的到底是不是那份认可的代码。”

Open Enclave 官方就把 enclave 应用分成两部分：不可信的 host 和可信的 enclave，这种划分非常适合教学解释。(openenclave.io)

第二步：发起远程证明

TEE 把自己的证明材料发给 attestation 服务。
这个服务会检查：

- 平台是不是可信
- 代码指纹是不是白名单中的
- 策略是否满足

- 是否符合当前安全要求

如果通过，就返回一个签名过的 token。
Azure Attestation 官方介绍中也明确提到，它会验证证据、评估策略、生成带签名的 token。(Microsoft Learn)

第三步：通过证明后领取密钥

这个 token 不是摆设，它的用处是：

- 去请求模型解密密钥
- 去请求事件签名密钥
- 去请求上传截图的访问密钥

也就是说：

不是谁启动了程序就能拿到密钥，而是只有“通过证明的可信程序”才能拿到。

这一步非常适合答辩，因为它把“可信环境”和“密钥管理”真正连起来了。
Azure Attestation 官方文档就强调 attestation token 可以供依赖方做进一步信任决策。(Microsoft Learn)

第四步：视频进入 TEE

摄像头采集到一帧视频后，不是直接给普通后端，而是先送进 TEE。

在 TEE 里面依次完成：

1. 解码图像
2. 人脸模糊
3. 可选的背景弱化
4. YOLO 检测
5. ByteTrack 追踪
6. 位移和静止时间计算
7. 风险判定
8. 事件摘要输出

TEE 外看不到完整原始帧，只能拿到“结果”。

第五步：只输出事件摘要

TEE 外部收到的不是视频，而是类似下面的 JSON：


```
{
  "event_type": "fall",
  "risk_level": "HIGH",
  "start_time": "2026-04-13T09:12:21+08:00",
  "end_time": "2026-04-13T09:12:29+08:00",
  "duration_sec": 8.0,
  "anon_person_id": "p03",
  "feature_summary": {
    "center_drop_ratio": 0.68,
    "aspect_ratio_change": 1.73,
    "stillness_window_sec": 6.2
  },
  "snapshot_type": "blurred",
  "snapshot_path": "/secure/event_snaps/evt_00128.jpg"
}
```


这就是**事件级输出**。

八、给学生一个更直观的例子

例子：老人跌倒事件

摄像头拍到老人摔倒。

如果没有 TEE：

- 视频会被普通后端接收
- 解码后的画面在普通内存里
- 其他高权限进程可能有机会读取
- 运维或调试人员也可能接触到完整画面

如果有 TEE：

- 视频先进入 TEE

- 在 TEE 内完成模糊和检测
- 外部只拿到：
    - “发生跌倒”
    - “高风险”
    - “持续 8 秒”
    - “匿名对象 p03”
    - “一张模糊后截图”

这样系统还能工作，但隐私暴露面更小。

九、差分隐私：给学生最容易懂的版本

1. 为什么图表也有风险

学生往往会觉得：“都没有原始视频了，图表还会泄露什么？”

举个例子。

Dashboard 上显示：

- 本周夜间异常 9 次
- 都发生在凌晨 2:00–3:00
- 最近 10 天持续上升

如果这个系统只服务一个老人，那这些统计图就已经在暴露这个人的夜间行为模式了。

所以即使没有视频，**长期聚合统计本身也可能泄露隐私**。差分隐私正是为这种“统计发布风险”设计的。(NIST Computer Security Resource Center)

---

2. 差分隐私最简单的理解

可以把它理解成：

在公布统计结果时，加一点“有控制的随机模糊”。

例如某天真实跌倒次数是 12，系统对外展示时可能显示成：

- 11
- 12
- 13

这样整体趋势还能看，但别人不容易从单次查询中推断得太精确。

---

3. 差分隐私不适合加在哪里

这点一定要强调。

不要把差分隐私加在实时检测主链路上。

也就是说，不要这样做：

- 实时跌倒判断加噪声
- 告警时间加噪声
- 单条事件详情加噪声

因为那会直接影响系统功能。

差分隐私最适合加在：

- 日统计
- 周统计
- 风险等级分布
- 夜间异常趋势
- 柱状图和饼图的数据

NIST 的指南也是围绕“差分隐私软件方案的保证如何评估”来展开，重点场景正是统计发布。(NIST Computer Security Resource Center)

十、差分隐私里最重要的三个词，学生必须懂

1. 隐私单元（privacy unit）

就是“到底想保护谁”。

在这个项目里，最合适的定义是：

- 一个老人，或者
- 一个家庭，或者
- 一个房间对应的一个监护对象

NIST 特别强调 privacy unit 必须定义清楚，否则隐私保证会说不明白。
(NIST 出版物)

---

2. epsilon（ε）

这是差分隐私里最常见的参数。

可以简单理解成：

- ε 小：保护更强，但结果更模糊
- ε 大：结果更准，但保护更弱

教学上可以直接告诉学生：

- ε = 0.5：保护偏强
- ε = 1.0：比较常见
- ε = 2.0：结果更准，但保护弱一些

这种比赛作品，统计图建议用 **0.5 到 1.0** 之间，比较稳妥。

---

3. 噪声机制

对计数类统计，最常见的是 **Laplace 机制**。
它的意思是：

- 先算真实值
- 再随机加一点噪声
- 噪声大小和 ε 有关系

学生不一定要背公式，但要知道本质：

结果不是乱改，而是按数学规则加噪声。

十一、系统里差分隐私该具体加在哪些接口

建议非常具体地告诉学生：

可以加的接口

比如：

- /api/stats/daily
- /api/stats/weekly
- /api/stats/risk_distribution
- /api/stats/night_trend

这些都属于“聚合统计发布”。

不要加的接口

比如：

- /api/event/realtime
- /api/event/detail
- /api/alert/push

这些是实时功能或单条事件，不适合加差分隐私。

十二、给学生一个最简单的差分隐私代码示例

下面这段最适合教学：

```
import numpy as np

def dp_count(true_count: int, epsilon: float, sensitivity: float =
1.0) -> int:
    noise = np.random.laplace(loc=0.0, scale=sensitivity / epsilon)
```

```
    noisy = true_count + noise
    return max(0, int(round(noisy)))
```

可以这样解释：

- true_count 是真实计数，比如 12 次跌倒
- epsilon 是隐私强度参数
- noise 是拉普拉斯噪声
- 最后把结果四舍五入，并且保证不小于 0

例如：

- 真实值 12
- ε = 0.8
- 加噪后可能是 11、12、13、10

这个结果用来画图通常还能接受。

十三、差分隐私在图表里怎么才不会“看起来很怪”

这也是学生最关心的。

1. 对低频数据做阈值隐藏

如果某天真实值只有 1 次或 2 次，加噪后图表容易很跳。
做法是：

- 小于某阈值就显示“<5”
- 或者直接显示“低频”

这样更稳。

---

2. 不按分钟统计

如果按分钟统计，噪声会让图非常乱。
建议只做：

- 日统计
- 周统计
- 月统计

---

3. 同一张图短时间内不要每次都重新加噪

不然学生刷新 10 次，平均一下又能逼近真实值。
所以要：

- 对同一用户、同一时段、同一图表结果做缓存
- 比如缓存 10 分钟或 1 小时

---

4. 不要在事件详情页加噪

详情页本来就是要给监护人看真实事件的，不适合搞“模糊统计”。

十四、把 TEE 和差分隐私合起来，形成一条完整链路

可以让学生牢牢记住下面这句：

TEE 管输入和计算过程，差分隐私管输出和展示结果。

更完整一点就是：

- **原始视频进入 TEE**
- **TEE 内完成匿名化、检测、追踪、风险判定**
- **TEE 只输出事件摘要**
- **普通后端保存事件摘要**
- **做统计图时再加差分隐私噪声**

这就是一条完整的隐私增强链路。

十五、项目里最适合采用的“学生版实施方案”

考虑到很多学生基础一般，建议分成两个层次来讲。

第一层：原型可实现版

这是 7 天内最现实的。

做法

不一定真的在这 7 天里上真实 SGX 或机密虚拟机，而是先把系统按 TEE **兼容思路** 拆分成两部分：

A. secure_core

只做最敏感的事：

- 接收原始帧
- 人脸模糊
- 检测
- 追踪
- 风险判定
- 生成事件摘要

B. 普通后端

只做不敏感的事：

- 登录
- 查询
- 图表
- 告警
- 权限管理

这样学生至少先把“可信边界”设计明白。

Open Enclave 官方就把应用拆成 host 和 enclave 两部分，这种思想和的软件分层是一致的。(openenclave.io)

第二层：正式部署增强版

如果以后继续做，可以接真实机密计算环境，例如：

- Open Enclave
- Azure Attestation
- 支持 TEE 的机密虚拟机或机密容器

Azure Attestation 官方说明中支持对多类 TEE 状态进行证明，包括 SGX、VBS enclave、TPM 相关环境以及 Azure 机密计算场景。(Microsoft Learn)

十六、给学生的“最小可做步骤”

下面这部分最适合直接给学生分工。

第 1 步：把代码拆成可信核心和普通服务

目录可以改成这样：

```
backend/
  app.py
  api/
  services/
  secure_core/
    enclave_api.py
    anonymizer.py
    detector.py
    tracker.py
    risk_engine.py
    event_builder.py
  dp_stats/
    mechanisms.py
    stats_service.py
    budget_manager.py
```

学生先不用懂真实 TEE，只要先理解：

- secure_core 是以后准备放进 TEE 的部分

- 其他部分都不应该直接接触原始视频

第 2 步：改通信方式

以前可能是前端把视频帧发给普通后端。
现在要改成：

- 原始帧只发给 secure_core
- 普通后端只接收 event summary

第 3 步：先做“逻辑版 attestation”

如果真 TEE 来不及，可以做一个教学版：

- secure_core 启动时计算代码哈希
- 后端检查这个哈希是否在白名单里
- 只有通过检查，才允许解密模型、开始服务

这当然不是完整硬件级证明，但很适合作为 attestation-compatible 原型来展示。

第 4 步：模型文件加密

让学生理解密钥管理。

做法可以是：

- 把 ONNX 模型文件先加密存储
- secure_core 启动并通过校验后，再读取解密密钥
- 解密后的模型只在内存里使用，不落盘

这样学生就明白：
“不是谁拿到模型文件就能直接用。”

第 5 步：实现事件摘要输出

要求 secure_core 最终输出统一格式的 JSON：

- 事件类型
- 风险等级
- 开始时间
- 结束时间
- 持续时长
- 匿名 person_id
- 模糊后截图路径

这样普通后端只和这个 JSON 打交道。

第 6 步：做统计层差分隐私

让一个学生单独负责 dp_stats 模块。

只对这些图表加噪：

- 每日事件数
- 每周事件数
- 风险等级分布
- 夜间异常趋势

先不要做太复杂的理论，只要做到：

- Laplace 噪声
- 非负截断
- 简单缓存
- 一个基础 ε 设置

就足够答辩。

最适合的是

- **TEE 保护实时处理**
- **差分隐私保护统计发布**

这条路最贴合的场景。

十八、给学生一些可直接查阅的资料

建议让学生重点看下面四类资料，不要一上来就看论文。

1. 机密计算/TEE 入门资料

Confidential Computing Consortium 白皮书
适合了解“机密计算到底保护什么”，尤其是“data in use”这个概念。(Confidential Computing Consortium)

2. 远程证明资料

Azure Attestation 官方文档
适合理解什么是 attestation、为什么要用 token、为什么要先验证环境再释放密钥。(Microsoft Learn)

3. TEE 开发抽象

Open Enclave SDK 官方站点 / 仓库
适合理解“host + enclave”的结构，不一定现在就写 C/C++，但它的架构思想特别适合教学。(openenclave.io)

4. 差分隐私入门资料

NIST SP 800-226
适合理解：

- 什么是差分隐私
- 什么是 privacy unit
- 什么是 ε
- 如何评估一个 DP 方案是不是说得清楚。(NIST Computer Security Resource Center)

十九、最适合本科生理解的一段总结

我们这个项目不是把所有数据都上传到平台再去做分析，而是先把最敏感的视频处理过程关进 TEE 这个“可信小房间”里，在里面完成匿名化、检测、追踪和风险判定，外部系统只接收事件结果。然后，为了防止长期统计图表暴露老人的行为模式，们再对图表这种聚合结果加入轻量差分隐私噪声。前者解决“视频怎么安全处理”，后者解决“统计结果怎么安全展示”。

二十、作品里最稳的表述方式

最后给一个适合写进报告和答辩的版本：

本作品采用“TEE + 轻量差分隐私”的双层隐私增强方案。
在计算侧，系统将视频解码、匿名化预处理、人体检测、多目标追踪、风险判定和事件摘要生成封装为可信计算核心，优先运行于 TEE 兼容环境中，以保护视频数据在使用过程中的机密性与完整性；在输出侧，系统仅对 Dashboard 等聚合统计接口引入轻量差分隐私扰动，用于降低长期统计结果对个体行为模式的反推风险，而不影响实时检测与告警处置。该方案实现了从“原始视频处理安全”到“统计发布安全”的分层隐私保护。

先做“TEE 兼容架构 + 原型级实现”，不必马上做成真正跑在本地 TEE 硬件上的生产级系统。
TEE/机密计算本质上依赖硬件支持的可信执行环境；Azure 这类云方案就是把这种硬件能力做成了机密虚拟机、机密容器等服务。Azure 官方明确说明，机密计算是通过**基于硬件、可证明（attested）的 TEE** 来保护“使用中的数据”，而机密虚拟机可以在**不修改代码**的情况下提供这种硬件边界。(Microsoft Learn)

所以现在有三条路，按“最适合比赛”排序是这样的：

第一条路：不买硬件，先做“TEE 兼容原型”

这是最推荐的。

意思不是假装自己有 TEE，而是：

- 先把系统结构改成“可信核心”和“普通业务层”两部分
- 原始视频只进入可信核心
- 普通后端只接收事件摘要
- 把这部分写成“未来可直接迁移到 Open Enclave / Confidential VM / Confidential Container 部署”

这样做的好处是：
不需要买硬件，也**不需要 7 天内折腾复杂底层环境**，但仍然可以把隐私计算逻辑讲清楚。

第二条路：不买本地硬件，租用云上的机密计算环境

如果真想做一个“更像真的 TEE”版本，又不想买设备，最现实的是：

- 租一个云上的 Confidential VM
- 或者用 Confidential Container

Azure 官方说明这类环境本身就是基于 TEE 的机密计算环境，而且有的场景可以做到**无需改代码**就把应用迁进去。(Microsoft Learn)

这条路的缺点是：

- 需要云账号和一定部署能力
- 比赛期间调环境比较花时间

- 不一定比当前原型收益更大

现在最该怎么做

给一个最明确的答案：

结论

现在不要买 TEE 硬件。

应该做的是：

第一步：把系统做成“TEE 兼容架构”

也就是软件层面先把边界分出来。

第二步：把“真正 TEE 部署”写成可升级方案

比如：

- Open Enclave
- Azure Attestation
- Confidential VM
- Confidential Container

第三步：把差分隐私真正落到统计接口上

这个是当前最容易真做出来的部分。

那“具体怎么实现”呢？

下面按**现有项目**来讲，不讲空话。

一、先别想硬件，先改的软件结构

现在要做的不是“买一个 TEE 盒子”，而是把系统拆成两块：

1. 可信核心 secure_core

这个模块以后准备放进 TEE。

它只做最敏感的事：

- 接收视频帧
- 解码图像
- 人脸模糊
- 背景弱化 / 前景提取
- YOLO 检测
- ByteTrack 追踪
- 风险判定
- 事件摘要生成

2. 普通服务层

这个模块不用看原始视频。

它只做：

- 用户登录
- 事件列表
- Dashboard 图表
- 告警通知
- 权限管理
- 系统设置
- 差分隐私统计

可以让学生把目录先改成这样

```
backend/
  app.py
  api/
  services/
  secure_core/
    enclave_api.py
```

```
    frame_ingest.py
    anonymizer.py
    detector.py
    tracker.py
    risk_engine.py
    event_builder.py
  dp_stats/
    mechanisms.py
    stats_service.py
    budget_manager.py
  db/
  frontend/
```

这一步的意义非常大，因为它等于先完成了**“可信边界设计”**。

二、先做“逻辑版 TEE”，不要一上来做真 TEE

这一步很多学生能做。

先定义规则：

- 原始帧**只能进入** secure_core
- 普通后端**不能直接接触**原始帧
- secure_core 对外只输出事件摘要 JSON

这样就形成了第一版“TEE 兼容模式”

虽然这时候还没跑在真正硬件 TEE 里，但的系统已经具备了：

- 敏感数据单独处理
- 对外最小输出
- 原始数据不进入业务层

这在比赛里完全讲得通。

三、应该先实现什么接口

1. secure_core 输入接口

只接收：

- 视频帧
- 摄像头流
- 风险阈值配置
- 模型版本信息
- 会话开始/结束命令

例如：

```
def process_frame(frame_bytes, session_id, config):
    ...
```

---

2. secure_core 输出接口

只返回：

- 事件类型
- 风险等级
- 开始时间
- 结束时间
- 持续时长
- 匿名 person_id
- 模糊后截图路径
- 特征摘要

例如：

```
{
  "event_type": "fall",
  "risk_level": "HIGH",
  "start_time": "2026-04-14T10:20:31+09:00",
  "end_time": "2026-04-14T10:20:39+09:00",
  "duration_sec": 8.0,
  "anon_person_id": "p02",
  "feature_summary": {
    "center_drop_ratio": 0.71,
    "aspect_ratio_change": 1.62,
    "stillness_window_sec": 5.8
  },
```

```
  "blurred_snapshot_path": "/events/evt_1021.jpg"
}
```

普通后端只和这个 JSON 打交道。

四、接下来做“伪 attestation”，而不是硬件 attestation

现在不买硬件，那就不要宣称“已经实现完整远程证明”。

应该做的是：

原型级证明流程

步骤 1

secure_core 启动时，计算核心代码和模型文件的哈希值。

步骤 2

后端维护一个白名单：

- 哪些代码哈希是认可的
- 哪些模型版本是认可的

步骤 3

只有当哈希匹配时，后端才允许：

- 加载模型
- 开启检测会话
- 写入事件数据库

这个流程不是官方意义上的硬件 attestation，但逻辑上和 attestation 很接近：

- 先证明“是谁”
- 再决定“给不给权限”

Azure Attestation 官方的核心流程就是：验证证据、评估策略、生成信任令牌。现在做的是一个教学/原型级的等价版本。(Microsoft Learn)

五、模型文件怎么保护

这一步很容易做，还特别加分。

不要把 ONNX 模型明文直接放在目录里

可以改成：

- 模型文件磁盘上加密保存
- secure_core 启动后，校验哈希
- 校验通过，才读密钥解密
- 解密后的模型只在内存里加载，不再落盘

为什么这样做

因为真正的机密计算场景里，一个重要能力就是：
只有被证明可信的执行环境才能拿到密钥。

Azure Attestation 官方也明确提到，attestation 结果可以给依赖方使用，常见用途就是后续做密钥释放。(Microsoft Learn)

六、如果以后真要上 TEE，怎么走

现在不用买硬件，但以后真想做成“真 TEE”版本，可以选下面两条路。

路线 A：云上机密虚拟机

这是最省事的“真 TEE”路线。

做法

- 把 secure_core 部署到 Azure Confidential VM
- 普通后端仍然部署在普通服务上

- 用 Azure Attestation 做远程证明

好处

- 不需要自己买硬件
- 官方已经把 TEE 环境准备好了
- 有 attestation 方案可用

Azure 官方说明，Confidential VM 是硬件强制隔离的机密执行边界，并且某些场景下可不改代码直接迁移。(Microsoft Learn)

路线 B：Open Enclave / 机密容器

这条更像研究/工程增强路线。

做法

- 把 secure_core 的核心敏感部分进一步封装成 enclave
- host 只负责 I/O、网络、数据库
- enclave 负责敏感内存计算
- 接 attestation 服务

Open Enclave 的定位就是给不同 TEE 提供统一的 enclave 开发抽象，非常适合把应用拆成 host + enclave 两部分。(Microsoft Learn)

现在最现实的实施步骤

给一个“7 天内可落地”的版本。

第 1 天：完成可信边界拆分

目标：

- 新建 secure_core
- 所有原始帧只能进 secure_core
- 普通后端不再直接处理原始图像

第 2 天：统一事件摘要输出

目标：

- 定义统一 JSON
- person_id 匿名化
- 只输出模糊后截图
- 默认不保存原始视频

第 3 天：做原型级哈希校验

目标：

- secure_core 启动时输出代码哈希
- 后端白名单校验
- 校验通过后才允许模型加载

第 4 天：模型文件加密

目标：

- ONNX 文件加密保存
- 检测启动前解密
- 解密模型只在内存里存在

第 5 天：做差分隐私统计模块

目标：

- 只处理 Dashboard 聚合图表
- 每日事件数、每周趋势、风险等级分布加噪
- 不影响实时检测

第 6 天：补两张图

一张图画：

- 摄像头 → secure_core → 事件摘要 → 后端

另一张图画：

- 数据库统计 → DP 加噪 → Dashboard

第 7 天：答辩口径统一

统一说法：

- 当前实现的是 **TEE-compatible prototype**
- 正式部署可迁移到 Confidential VM / Open Enclave
- 已经真正落地的是统计发布层的差分隐私

差分隐私这部分现在可以真做

相比 TEE，差分隐私是**现在最容易真正实现**的。

该怎么做

只对这些接口加噪：

- /api/stats/daily
- /api/stats/weekly
- /api/stats/risk_distribution
- /api/stats/night_trend

代码示例

```
import numpy as np

def dp_count(true_count: int, epsilon: float = 0.8, sensitivity:
float = 1.0) -> int:
    noise = np.random.laplace(0.0, sensitivity / epsilon)
    return max(0, int(round(true_count + noise)))
```

规则

- 低频值显示 <5
- 同一图表结果缓存 10 分钟
- 只对聚合统计加，不对单条事件详情加

NIST 强调差分隐私方案里，privacy unit、参数设置和后处理方式都要说明清楚；它还特别指出，像“把负值截断为 0”这种后处理虽然不破坏 DP 保证，但会影响统计偏差。(Microsoft Learn)

答辩时该怎么说最稳

可以直接这样回答老师或评委：

们当前原型并没有要求额外采购本地 TEE 硬件。考虑到比赛周期和工程成本，本作品先实现了 TEE 兼容的软件架构：将原始视频接收、匿名化、检测、追踪和风险判定封装为独立的可信计算核心，普通业务层只处理事件摘要结果。这样做的目的是先把敏感数据边界与最小暴露原则落实到系统结构中。
在正式部署层面，该可信核心可进一步迁移到支持硬件级 TEE 的环境中，例如云上的 Confidential VM 或基于 Open Enclave 的机密计算环境，并结合远程证明完成密钥释放与可信启动。
同时，们已经在统计发布层真正实现了轻量差分隐私，仅对日/周事件统计和风险分布图表加噪，避免长期聚合结果泄露个体行为模式。

这个回答比较稳，不夸大，也不显得“只停留在概念”。

目标不是假装已经做了真正的硬件级 TEE，而是把系统先改造成：

现在就能运行、能答辩、能写进报告，而且以后可以平滑迁移到真正 TEE 环境。

先把定位说清楚：

Open Enclave 官方把 TEE 应用描述为两部分：**不可信的 host** 和 **可信的 enclave**；CCC 也把机密计算的核心描述为保护“使用中的数据”；Azure Attestation 的作用则是远程验证平台和二进制是否可信，并返回可供依赖方使用的 attestation token。现在不买硬件，所以先做的是**这种边界思想的软件化原型**，而不是声称已经实现硬件级 enclave。(GitHub)

---

1. 这套方案到底要实现什么

现在要实现的，不是“真正的 TEE”，而是：

TEE-compatible prototype，也就是 TEE 兼容原型。

它要满足四个要求：

第一，原始视频帧不再进入普通业务后端。
第二，所有最敏感的处理都集中到一个独立的“可信核心”里。
第三，普通后端只能接收事件摘要，而不是完整视频或完整中间特征。
第四，将来如果要迁移到 Open Enclave、Confidential VM 或 Azure Attestation，只需要替换“可信核心”的运行环境，而不必重写整个系统。
Open Enclave 和 Azure 机密计算文档都强调，这类系统的关键是先划清 host 与可信执行部分的边界。(GitHub)

所以这套方案的本质是：

先做“软件边界正确”，再谈“硬件边界增强”。

---

2. 先理解要改造成什么样

把现在的系统想成两层。

原来的方式大概是：

前端/摄像头
→ 普通后端
→ 检测
→ 追踪
→ 风险判定
→ 数据库
→ 图表

要改成：

前端/摄像头
→ secure_core（未来可迁移到 TEE）
→ 事件摘要
→ 普通后端
→ 数据库 / 图表 / 告警

这里最关键的变化是：

普通后端不再碰原始视频。

这一点非常重要，因为机密计算的核心并不是“系统用了某个高深名词”，而是**敏感数据的处理边界真的被隔离出来了**。CCC 对 confidential computing 的说明也是围绕“data in use”保护展开的。(Confidential Computing Consortium)

---

3. 第一步：先定义“可信核心”边界

这一步最重要，而且必须先做。

要在项目里新建一个独立模块，名字就叫：

secure_core

它以后就是准备迁移到 TEE 的部分。
现在虽然它还只是普通进程，但要**按 enclave 的思路来约束它**。

3.1 哪些模块必须放进 secure_core

应该把下面这些东西都搬进去：

第一，视频帧接收与解码。
因为图像一旦解码出来，就是完整隐私数据。

第二，人脸模糊、背景弱化、前景提取。
因为匿名化之前仍然是高敏图像。

第三，检测与追踪。
包括 YOLO 推理、ByteTrack、多帧轨迹维护、位移计算等。

第四，风险判定。
包括跌倒连续帧、静止窗口、夜间异常等逻辑。

第五，事件摘要构造。
包括生成匿名 person_id、事件时长、风险等级、模糊截图路径等。

这一拆法和 Open Enclave 的“host + enclave”思想是一致的：敏感逻辑放可信侧，普通 I/O 和业务逻辑放 host 侧。(GitHub)

3.2 哪些模块必须留在普通后端

普通后端保留：

登录认证、页面查询、统计图表、事件列表、告警通知、系统设置、权限管理、差分隐私统计模块。

原因很简单：
这些功能不需要看到原始视频，只需要看到“结果”。

---

4. 第二步：先改项目目录

这一部分可以直接让学生照着改。

建议目录先改成这样：

```
backend/
  app.py
  api/
    auth_api.py
    event_api.py
    stats_api.py
    config_api.py
```

```
  services/
    alert_service.py
    event_service.py
  secure_core/
    __init__.py
    enclave_api.py
    session_manager.py
    frame_ingest.py
    decoder.py
    anonymizer.py
    detector.py
    tracker.py
    feature_extractor.py
    risk_engine.py
    event_builder.py
    integrity.py
    model_loader.py
  dp_stats/
    __init__.py
    mechanisms.py
    stats_service.py
    budget_manager.py
  db/
  storage/
  frontend/
```

这里的设计思想是：

secure_core/ 代表“未来 enclave 化的逻辑”。
api/ + services/ 代表普通业务层。
dp_stats/ 单独负责统计发布层的差分隐私。

这样做的好处是，从代码结构上就已经体现了“可信边界”。这和 Open Enclave 官方强调的 host / enclave 分区是一致的。(GitHub)

---

5. 第三步：先定义数据流规则

这一部分必须写成团队开发规范，不然学生改着改着又把视频帧传回普通后端了。

要明确规定下面三条规则：

5.1 规则一：原始视频帧只能进入 secure_core

普通 API 层不允许保存、不允许转发、不允许日志打印原始帧。

也就是说，像下面这种写法以后都不允许：

```
# 错误示意：普通后端直接拿原始图像
frame = request.files["frame"].read()
result = detect(frame)
save_raw_frame(frame)
```

应该改成：

```
# 正确示意：普通后端只转交给 secure_core
frame = request.files["frame"].read()
event_summary = secure_core.process_frame(frame, session_id, config)
return jsonify(event_summary)
```

5.2 规则二：secure_core 对外只返回“事件摘要”

它不返回原始帧，不返回完整轨迹历史，不返回完整人体框序列。

5.3 规则三：数据库不落原始视频

默认只落：

- 事件类型
- 风险等级
- 开始/结束时间
- 匿名 person_id
- 特征摘要
- 模糊后截图路径

这样做的目的，就是先在软件层实现“最小暴露原则”。

---

6. 第四步 :定义 secure_core 的输入和输出接口

这是整个实现里最关键的工程步骤之一。

6.1 输入接口怎么设计

secure_core 最多只应该接收这些输入：

- frame_bytes：当前帧字节流
- session_id：本次检测会话 ID
- config：检测配置
- timestamp：当前帧时间戳

可以先定义一个统一入口：

```
def process_frame(
    frame_bytes: bytes,
    session_id: str,
    timestamp_ms: int,
    config: dict
) -> dict:
    """
    输入原始帧，输出事件摘要。
    """
    ...
```

如果是视频文件处理，则可以额外做：

```
def process_video(
    video_path: str,
    session_id: str,
    config: dict
) -> list[dict]:
    """
    输入本地视频路径，输出事件摘要列表。
    """
    ...
```

6.2 输出接口怎么设计

输出一定要尽量瘦身。建议结构固定成：

```
{
  "session_id": "sess_20260414_001",
  "event_id": "evt_20260414_017",
  "event_type": "fall",
  "risk_level": "HIGH",
```

```
  "timestamp_start": "2026-04-14T10:20:31+09:00",
  "timestamp_end": "2026-04-14T10:20:39+09:00",
  "duration_sec": 8.0,
  "anon_person_id": "p02",
  "feature_summary": {
    "center_drop_ratio": 0.71,
    "aspect_ratio_change": 1.62,
    "stillness_window_sec": 5.8
  },
  "blurred_snapshot_path": "/events/evt_1021.jpg",
  "core_hash": "sha256:....",
  "model_version": "fall_v1.2"
}
```

这里特意加上 core_hash 和 model_version，是为了给后面“原型级 attestation”做准备。

---

7. 第五步：拆 secure_core 内部模块

下面按学生能实现的方式，把每个模块该做什么都写清楚。

7.1 frame_ingest.py

作用是接收帧，做最基本的输入校验。

要做的具体事情是：

检查字节流长度是否合理。
限制最大帧尺寸。
拒绝异常格式。
给每一帧挂上 session_id 和时间戳。
不要在这里做日志打印原图。

示意代码：

```
class FrameIngestor:
    def validate_frame(self, frame_bytes: bytes) -> None:
        if not frame_bytes:
            raise ValueError("empty frame")
        if len(frame_bytes) > 5 * 1024 * 1024:
            raise ValueError("frame too large")
```

```
    def ingest(self, frame_bytes: bytes, session_id: str,
timestamp_ms: int):
        self.validate_frame(frame_bytes)
        return {
            "session_id": session_id,
            "timestamp_ms": timestamp_ms,
            "frame_bytes": frame_bytes,
        }
```

7.2 decoder.py

作用是把 JPEG / PNG / H.264 帧变成内存中的图像矩阵。

这里有一个重要原则：

解码后的图像对象不应该传出 secure_core。

例如：

```
import cv2
import numpy as np

class Decoder:
    def decode_jpeg(self, frame_bytes: bytes):
        arr = np.frombuffer(frame_bytes, dtype=np.uint8)
        img = cv2.imdecode(arr, cv2.IMREAD_COLOR)
        if img is None:
            raise ValueError("decode failed")
        return img
```

7.3 anonymizer.py

作用是匿名化处理。

已经有人脸模糊模块，那就搬到这里。
这里要做三层处理：

第一层，人脸模糊。
第二层，可选的背景弱化。
第三层，可选的轮廓化/前景化。

建议做成分级配置：

```
{
  "face_blur": true,
  "background_mask": false,
  "contour_only": false,
  "save_blurred_snapshot": true
}
```

这样答辩时就能说：
系统支持多档匿名化强度。

7.4 detector.py

作用是人体检测或跌倒姿态检测。

这个模块内部可以继续用现有的 ONNX 推理代码。
关键要求只有两个：

第一，模型加载必须改到 model_loader.py，不要在外部随便直接加载。
第二，检测结果只作为 secure_core 内部对象流转，不直接对外输出。

7.5 tracker.py

作用是追踪和 ID 分配。

这里一定要注意：

外部不要直接看到原始追踪 ID。
外部只能看到匿名 person_id。

原因是：
真实追踪 ID + 长期轨迹可能被用来重建个人行为模式。

7.6 feature_extractor.py

作用是把底层检测结果变成更紧凑的结构化特征。

比如：

- 目标框中心点变化
- 长宽比变化
- 位移均值
- 静止时长
- 夜间活动次数

这里是以后最适合 enclave 化的部分之一，因为它正好卡在“原始信息”与“高层事件”之间。

7.7 risk_engine.py

作用是做规则融合。

这里不变，只是位置变了：
从普通后端搬到 secure_core。

例如：

- 连续 5 帧 fallen → MEDIUM
- fallen 持续超过 1 秒 → HIGH
- 静止超过 30 秒 → LOW
- 夜间静止异常 → MEDIUM

7.8 event_builder.py

作用是把风险结果压缩成对外输出的事件摘要。

一定要在这里做两件事：

第一，把内部追踪 ID 映射成匿名 ID。
第二，只挑最必要的字段对外输出。

示意代码：

```
class EventBuilder:
    def build_summary(self, event, feature_summary,
blurred_snapshot_path, core_hash, model_version):
        return {
            "session_id": event.session_id,
            "event_id": event.event_id,
            "event_type": event.event_type,
            "risk_level": event.risk_level,
            "timestamp_start": event.start_time,
            "timestamp_end": event.end_time,
            "duration_sec": event.duration_sec,
            "anon_person_id": f"p{event.person_local_index:02d}",
            "feature_summary": feature_summary,
            "blurred_snapshot_path": blurred_snapshot_path,
            "core_hash": core_hash,
```

```
        "model_version": model_version,
    }
```

---

8. 第六步：做“原型级 attestation-compatible”流程

因为现在没有 TEE 硬件，所以不要做真正硬件 attestation。
但完全可以做一个**逻辑等价版**，用来表达“可信启动”和“可信放行”。

Azure Attestation 的正式做法是：验证平台和二进制完整性，成功后返回 token；实例所有者还可以用 policy 定义要检查什么。现在做的是一个软件原型版：不验证硬件，而是验证代码和模型的完整性，再决定是否放行。
(Microsoft Learn)

8.1 具体步骤

步骤 1：启动时计算核心代码哈希

在 integrity.py 里，对 secure_core/ 关键文件做 SHA-256。

例如：

```
import hashlib
from pathlib import Path

CORE_FILES = [
    "secure_core/anonymizer.py",
    "secure_core/detector.py",
    "secure_core/tracker.py",
    "secure_core/risk_engine.py",
    "secure_core/event_builder.py",
]

def sha256_file(path: str) -> str:
    h = hashlib.sha256()
    with open(path, "rb") as f:
        h.update(f.read())
    return h.hexdigest()
```

```
def compute_core_hash() -> str:
    h = hashlib.sha256()
    for p in CORE_FILES:
        h.update(sha256_file(p).encode())
    return h.hexdigest()
```

步骤 2：模型文件单独计算哈希

对 ONNX 模型也算哈希。

```
def compute_model_hash(model_path: str) -> str:
    return sha256_file(model_path)
```

步骤 3：后端维护白名单

例如在配置文件里写：

```
{
  "trusted_core_hashes": [
    "abc123...",
    "def456..."
  ],
  "trusted_model_hashes": [
    "mhash001..."
  ]
}
```

步骤 4：启动时检查哈希

只有当 core hash 和 model hash 都在白名单中，系统才允许 secure_core 进入“可服务状态”。

```
def verify_integrity(core_hash: str, model_hash: str, whitelist:
dict) -> bool:
    return (
        core_hash in whitelist["trusted_core_hashes"]
        and model_hash in whitelist["trusted_model_hashes"]
    )
```

步骤 5：校验失败则拒绝启动

校验不通过时，不允许：

- 加载模型
- 开启新会话
- 写入事件数据库

步骤 6：把哈希写入每条事件摘要

这样普通后端可以知道：
这条事件是由哪个“可信核心版本”生成的。

这虽然不是官方硬件 attestation，但逻辑上已经模拟了三件关键事情：

- 验证“运行的是谁”
- 验证“模型是不是那份模型”
- 只有通过验证才放行

这就是报告里所谓的 attestation-compatible startup flow。

---

9. 第七步：实现模型文件保护

真正 TEE 系统里，常见做法是“证明通过后释放密钥”。
现在没有硬件，也可以先做一个简化版：**模型文件磁盘加密 + 启动校验后解密。**

9.1 具体做法

步骤 1：把 ONNX 模型加密保存

不要直接把 fall_detection_v1.onnx 明文放在仓库里。

可以存成：

fall_detection_v1.onnx.enc

步骤 2：在本地保存一个开发密钥

比赛原型阶段可以先用环境变量：

```
export MODEL_ENC_KEY="your-dev-key"
```

步骤 3：启动时先校验 integrity

只有 integrity 通过，才调用模型解密函数。

步骤 4：解密后只在内存里加载

不要把明文模型再写回磁盘。

示意代码：

```
from cryptography.fernet import Fernet

class ModelLoader:
    def __init__(self, key: bytes):
        self.key = key

    def decrypt_model_bytes(self, enc_path: str) -> bytes:
        with open(enc_path, "rb") as f:
            enc = f.read()
        cipher = Fernet(self.key)
        return cipher.decrypt(enc)

    def load_model_in_memory(self, enc_path: str):
        model_bytes = self.decrypt_model_bytes(enc_path)
        # 这里再交给 onnxruntime 从内存加载，或临时文件方式加载后立即
删除
        return model_bytes
```

9.2 为什么这一步很重要

因为这一步已经体现了“可信环境放行后才拿到关键资产”的思想。
将来如果真的迁移到 Azure Attestation 或 Confidential VM，这个“放行条件”就可以从本地白名单校验升级为正式 attestation token 驱动。Azure Attestation 文档明确说明 attestation token 可供依赖方用于进一步授权决策。(Microsoft Learn)

---

10. 第八步：限制普通后端的权限

光有 secure_core 还不够，还得反过来限制普通后端。

10.1 普通后端不应拥有的能力

普通后端不能：

- 保存原始帧
- 调试打印原始图像
- 直接读取模型明文
- 读取完整追踪轨迹序列
- 导出未模糊截图

10.2 普通后端只应拥有的能力

普通后端只能：

- 接收事件摘要
- 接收模糊后截图路径
- 写事件表
- 查事件表
- 推告警
- 做统计图

这一步建议在代码里做成“硬限制”。

比如，secure_core 输出对象明确只包含允许字段。
普通后端 DTO 不定义任何 raw_frame、full_track_history 之类字段。

---

11. 第九步：定义数据库层只存什么

数据库也要跟着改。

11.1 事件表建议字段

建议保留：

- event_id
- session_id
- event_type
- risk_level
- timestamp_start
- timestamp_end
- duration_sec
- anon_person_id
- feature_summary_json
- blurred_snapshot_path
- core_hash
- model_version

11.2 不要存的字段

不要存：

- raw_frame_path
- unblurred_face_path
- full_track_points
- full_bbox_sequence

因为一旦把这些字段又存起来，就相当于绕过了整个“可信边界”设计。

---

12. 第十步：做一条真正跑得通的处理链

下面是应该一步一步真正跑通的流程。

12.1 会话启动

普通后端调用：

```
core_info = secure_core.start_session(session_id, config)
```

start_session() 内部做：

1. 计算 core hash
2. 计算 model hash
3. 检查白名单
4. 通过后加载模型

5. 初始化追踪器与状态缓存
6. 返回：

```
{
  "session_id": "sess_001",
  "core_hash": "abc123...",
  "model_hash": "mhash001...",
  "status": "trusted-ready"
}
```

12.2 帧处理

每来一帧：

1. frame_ingest 校验
2. decoder 解码
3. anonymizer 处理
4. detector 检测
5. tracker 追踪
6. feature_extractor 算特征
7. risk_engine 判风险
8. event_builder 输出事件摘要

12.3 后端接收摘要

普通后端收到的内容只类似于：

```
{
  "event_type": "fall",
  "risk_level": "HIGH",
  "duration_sec": 8.0,
  "anon_person_id": "p02",
  "blurred_snapshot_path": "/events/evt_1021.jpg",
  "core_hash": "abc123..."
}
```

然后：

- 写数据库
- 推告警
- 更新前端页面

12.4 会话结束

结束时由 secure_core.close_session(session_id) 清理：

- 轨迹缓存
- 临时特征缓存
- 内存中的解码图像
- 临时模糊帧缓冲

这一步也很重要，因为“敏感数据生命周期最短化”本身就是好设计。

---

13. 第十一步：加入审计与日志，但别泄露隐私

最好加审计日志，但要注意日志本身不能变成新的隐私泄露源。

13.1 可以记录什么

可以记录：

- session 开始/结束
- core hash
- model version
- 是否通过 integrity 校验
- 事件条数
- 告警条数

13.2 不要记录什么

不要记录：

- 原始帧内容
- 明文人脸框坐标全集
- 全量轨迹点
- 原始视频文件路径

13.3 典型日志

```
[2026-04-14 10:20:01] secure_core start session=sess_001
core_hash=abc123 model_hash=mhash001 trusted=yes
[2026-04-14 10:20:39] event generated session=sess_001 type=fall
risk=HIGH anon_person=p02
[2026-04-14 10:21:10] secure_core close session=sess_001 events=1
```

---

15. 第十三步：答辩时该怎么准确表述

这一点很重要，避免说大了。

现在可以说：

本作品当前实现的是 TEE 兼容的软件架构原型。们已将视频解码、匿名化、检测、追踪、风险判定和事件摘要生成封装为独立的可信计算核心，原始视频不再进入普通业务后端。系统通过原型级完整性校验流程对可信核心代码和模型进行哈希验证，只有校验通过后才允许模型加载和会话启动。该设计在不依赖额外 TEE 硬件的前提下，先实现了与 enclave 设计一致的敏感数据边界约束，后续可平滑迁移到 Open Enclave、Confidential VM 或 Azure Attestation 支持的正式机密计算环境。

这个说法和 Open Enclave 的 host/enclave 分区、Azure Attestation 的验证后放行思路是一致的。(GitHub)

---

16. 第十四步：这套方案现在做到了什么，没做到什么

16.1 现在已经做到的

现在这套 TEE 兼容方案真正做到的是：

- 原始视频不进入普通业务层
- 敏感计算被集中封装
- 对外只暴露事件摘要
- 启动时做核心完整性校验
- 模型资产做了受控加载

- 以后可以迁移到正式 TEE

16.2 现在还没做到的

现在还没做到的是：

- 硬件级内存隔离
- 正式 quote / report 级远程证明
- 正式 attestation token
- 真正由硬件保障的 enclave 内存保护

这些只有以后迁移到真实 TEE 环境时才会具备。Azure Attestation 和 Open Enclave 文档都表明，正式 attestation 依赖底层受支持的 TEE 环境。(Microsoft Learn)

所以现在做的是：

先把架构做对，别把硬件当第一步。

---

17. 最后给一个最简执行清单

如果只想抓重点，现在就做这 8 件事：

1. 新建 secure_core/。
2. 把解码、匿名化、检测、追踪、风险判定都搬进去。
3. 普通后端只调用 secure_core.process_frame()。
4. 对外只返回事件摘要 JSON。
5. 实现 core_hash + model_hash。
6. 加白名单校验，不通过就拒绝启动。
7. 模型文件改为加密存储、运行时受控加载。
8. 数据库和日志只存摘要，不存原始视频。

这 8 件事做完，就已经有一套**真正能说清楚、能展示、能写报告的 TEE 兼容实现方案**了。

下面 “轻量差分隐私实现方案”。按这条线来讲：

先理解放在哪一层 → 再定保护对象和参数 → 再改接口 → 再写代码 → 再测试 → 再避免常见坑。

先给一个最核心的结论：

这次只把差分隐私加在“统计发布层”，不要加在实时检测、单条事件详情、即时告警上。
这和差分隐私的典型用途是吻合的：NIST 把差分隐私描述为一种量化“当某个个体数据出现在数据集中时，系统输出会泄露多少隐私风险”的框架；OpenDP 也把 Laplace 机制作为对计数、求和、均值这类聚合结果做私有化的常见方法。(NIST 出版物)

一、先让负责的同学明白：这块到底做什么

系统里现在已经有很多真实业务：

- 跌倒检测
- 长时间静止检测
- 夜间异常检测
- 事件列表
- Dashboard 图表
- 风险等级分布
- 告警记录

差分隐私这次只负责一件事：

当系统把“统计图表”展示给用户时，不直接给真实计数，而是在真实计数上加一点受控制的随机噪声。

这样做的目标不是“把结果搞假”，而是：

- 大趋势还在
- 图表还能看
- 但别人更难从统计结果里反推出某个老人具体发生了什么

NIST SP 800-226 把评估差分隐私方案时最重要的内容之一概括为：要先明确威胁模型、查询模型、暴露面、算法正确性和隐私预算，而不是只看“有没有加噪声”。(NIST 出版物)

二、项目里，哪些地方该加，哪些地方坚决不要加

应该加的地方

只加在**聚合统计接口**上，比如：

- 今日事件总数
- 本周跌倒次数
- 高风险事件数
- 风险等级分布
- 夜间异常趋势
- 按天统计柱状图
- 按周统计折线图

这些本质上都是“计数查询”或“分组计数查询”，非常适合用 Laplace 机制。OpenDP 的文档明确把 Laplace 机制描述为常用于 sum、count、mean 这类聚合发布的机制。(OpenDP)

不要加的地方

不要加在：

- 实时视频检测结果
- 单条事件详情页
- 即时告警推送
- 家属查看某条具体异常记录
- 风险引擎内部阈值判断

因为这些地方要求“真实、及时、可处置”。
如果在这些环节加噪声，会直接破坏系统功能，而不是增强隐私。

三、先定“到底在保护谁”

这一步是很多初学者最容易忽略的，但 NIST 特别强调必须先定义清楚。这个东西叫 privacy unit，也就是“隐私单元”。(NIST 出版物)

在项目里，建议这样定义

最稳妥的选择是：

一个老人 / 一个被监护对象 = 一个 privacy unit

也就是说，希望做到的是：

就算某个老人是否出现在统计数据里发生变化，最终图表结果也不应该有太大可区分差异。

这个定义比“每条事件是一单位”更强，也更符合的答辩逻辑。

为什么不建议用“每条事件”做隐私单元

因为那样保护太弱了。
例如某个老人一天内发生 3 次夜间异常，如果按事件级来保护，仍可能通过多次统计把个人模式推出来。

四、先定的差分隐私范围

不是全系统差分隐私，而是“统计发布层轻量差分隐私”。

所以这次的范围应该写死：

1. 只处理 Dashboard 和 Analysis 页的聚合统计。
2. 只对计数类、分布类、趋势类结果加噪。
3. 只做读取时加噪，不改数据库原始事件数据。
4. 只在返回前端前加噪，不在存储阶段加噪。

这个思路和常见 DP 部署方式是一致的：原始数据保留在受控系统内，DP 机制应用在“对外发布的查询结果”上。(NIST 出版物)

五、先选最简单、最稳的机制：Laplace 机制

对现在的场景,建议不要一上来讲 Gaussian、Rényi DP、advanced composition 这些东西。
先把最实用的做好。

只用一个机制就够了

Laplace 机制

它适合处理：

- count
- grouped count
- histogram bin count
- daily total
- weekly total

OpenDP 把 Laplace 机制明确列为 DP 生态中最常用的聚合私有化算法之一。(OpenDP)

先用最简单的公式理解

如果真实计数是 c，返回值不是直接给 c，而是：

c + 噪声

这个噪声来自 Laplace 分布，尺度通常写成：

scale = sensitivity / epsilon

其中：

- sensitivity：敏感度
- epsilon：隐私预算参数

对这种简单计数查询，**敏感度通常先取 1**。
也就是说，一个 privacy unit 的加入或删除，最多让计数变化 1。

六、先定参数，别让学生自己瞎试

初学者最怕“ε 怎么选”。

这次做的是轻量统计发布，建议这样定：

参数建议

单个接口默认

- epsilon = 0.8
- sensitivity = 1.0

如果某个图表特别重要，想更准一点

- epsilon = 1.0

如果某个图表隐私更敏感

- epsilon = 0.5

对于的比赛项目，不建议把 epsilon 设得太大。
太大就接近没保护；太小图表又太乱。
0.5 ~ 1.0 是一个比较好讲、也比较好用的范围。NIST SP 800-226 的重点不是给出固定“标准 ε”，而是强调要明确参数、威胁模型、查询模型和组合方式。(NIST 出版物)

七、实现思路：不要改数据库，做一个单独的 dp_stats 模块

这一步最适合工程实现。

应该新建一个模块，比如：

backend/

```
dp_stats/
  __init__.py
  mechanisms.py
  budget_manager.py
  stats_service.py
  cache_service.py
  schemas.py
```

各文件职责如下：

1. mechanisms.py

只负责“怎么加噪声”。

2. budget_manager.py

只负责“每个查询消耗多少 ε，今天总共用了多少 ε”。

3. stats_service.py

只负责“先查真实统计，再调用机制，再输出前端能用的数据”。

4. cache_service.py

防止用户反复刷新同一张图，把噪声平均掉。

5. schemas.py

定义 API 返回结构。

八、第一步代码：先写最小可用的 Laplace 机制

先给一个**不依赖第三方 DP 库**、最容易读懂的版本。
适合学生先理解逻辑。

```python
# backend/dp_stats/mechanisms.py
from __future__ import annotations

import math
import secrets
from dataclasses import dataclass

import numpy as np


@dataclass(frozen=True)
class DPParams:
    epsilon: float
    sensitivity: float = 1.0

    def __post_init__(self):
        if self.epsilon <= 0:
            raise ValueError("epsilon must be > 0")
        if self.sensitivity <= 0:
            raise ValueError("sensitivity must be > 0")


def sample_laplace(scale: float) -> float:
    """
    生成 Laplace(0, scale) 噪声。
    这里直接使用 numpy 的 laplace 分布。
    """
    if scale <= 0:
        raise ValueError("scale must be > 0")
    return float(np.random.laplace(loc=0.0, scale=scale))


def privatize_count(true_count: int, params: DPParams) -> int:
    """
    对单个计数值做差分隐私处理。
    """
    if true_count < 0:
        raise ValueError("true_count must be >= 0")

    scale = params.sensitivity / params.epsilon
    noisy_value = true_count + sample_laplace(scale)

    # 后处理：四舍五入，并且不允许负数
    return max(0, int(round(noisy_value)))
```

```
def privatize_counts_dict(true_counts: dict[str, int], params:
DPParams) -> dict[str, int]:
    """
    对一个类别分布做逐项私有化。
    例如 {'fall': 12, 'still': 5, 'night': 8}
    """
    result: dict[str, int] = {}
    for key, value in true_counts.items():
        result[key] = privatize_count(value, params)
    return result
```

这段代码已经可以处理：

- 总事件数
- 每种事件类型计数
- 风险等级分布
- 每天统计值

九、第二步代码：做预算管理，不然会被反复查询“刷穿”

NIST 强调，差分隐私不能只看单次查询，要看整体查询模型和累计隐私损失。(NIST 出版物)

所以要实现一个最简单的预算管理器。

```
# backend/dp_stats/budget_manager.py
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import date
from typing import Dict, Tuple


@dataclass
class BudgetManager:
    """
    简化版预算管理器：
```

```
    按 user_id + 日期 记录当天已消耗的 epsilon。
    """
    daily_limit: float = 3.0
    usage: Dict[Tuple[str, date], float] =
field(default_factory=dict)

    def can_spend(self, user_id: str, epsilon: float, today: date) ->
bool:
        spent = self.usage.get((user_id, today), 0.0)
        return spent + epsilon <= self.daily_limit

    def spend(self, user_id: str, epsilon: float, today: date) ->
None:
        spent = self.usage.get((user_id, today), 0.0)
        new_value = spent + epsilon
        if new_value > self.daily_limit:
            raise ValueError("privacy budget exceeded")
        self.usage[(user_id, today)] = new_value

    def get_spent(self, user_id: str, today: date) -> float:
        return self.usage.get((user_id, today), 0.0)
```

现在怎么用

建议：

- 每张图表查询消耗 epsilon = 0.8
- 每个用户每天总预算 3.0

这样一天大约允许查 3 到 4 张图，已经够展示了。

如果超预算，可以：

- 返回缓存结果
- 或提示“今日统计已启用缓存保护”

十、第三步代码：加缓存，防止用户反复刷新逼近真实值

这是新手很容易忘掉的坑。

如果同一张图每刷新一次都重新采样噪声，那么用户连刷 50 次取平均，可能会逼近真实值。
所以必须做缓存。

```
# backend/dp_stats/cache_service.py
from __future__ import annotations

from dataclasses import dataclass, field
from datetime import datetime, timedelta
from typing import Any, Dict, Tuple


@dataclass
class CacheEntry:
    value: Any
    expires_at: datetime


@dataclass
class DPResultCache:
    ttl_minutes: int = 10
    store: Dict[Tuple[str, str, str], CacheEntry] =
field(default_factory=dict)

    def get(self, user_id: str, query_name: str, query_key: str, now:
datetime):
        key = (user_id, query_name, query_key)
        entry = self.store.get(key)
        if entry and entry.expires_at > now:
            return entry.value
        return None

    def set(self, user_id: str, query_name: str, query_key: str,
value: Any, now: datetime):
        key = (user_id, query_name, query_key)
        self.store[key] = CacheEntry(
            value=value,
```

```
            expires_at=now + timedelta(minutes=self.ttl_minutes),
        )
```

这里三个 key 是什么

- user_id：谁在查
- query_name：查的是哪个图表
- query_key：查的时间范围等条件

例如：

- query_name = "daily_counts"
- query_key = "2026-04-01_to_2026-04-14"

十一、第四步代码：把真实统计包装成“私有化统计服务”

这是学生最应该直接上手改的地方。

```
# backend/dp_stats/stats_service.py
from __future__ import annotations

from dataclasses import dataclass
from datetime import date, datetime
from typing import Dict, List

from .budget_manager import BudgetManager
from .cache_service import DPResultCache
from .mechanisms import DPParams, privatize_count,
privatize_counts_dict


@dataclass
class StatsService:
    budget_manager: BudgetManager
    cache: DPResultCache

    def get_private_total_count(
        self,
```

```
        *,
        user_id: str,
        query_key: str,
        true_count: int,
        epsilon: float,
        now: datetime,
    ) -> dict:
        cached = self.cache.get(user_id, "total_count", query_key,
now)
        if cached is not None:
            return {
                "value": cached,
                "source": "cache",
            }

        today = now.date()
        if not self.budget_manager.can_spend(user_id, epsilon,
today):
            raise ValueError("privacy budget exceeded")

        params = DPParams(epsilon=epsilon, sensitivity=1.0)
        noisy_count = privatize_count(true_count, params)

        self.budget_manager.spend(user_id, epsilon, today)
        self.cache.set(user_id, "total_count", query_key,
noisy_count, now)

        return {
            "value": noisy_count,
            "source": "fresh_dp",
        }

    def get_private_distribution(
        self,
        *,
        user_id: str,
        query_key: str,
        true_counts: Dict[str, int],
        epsilon: float,
        now: datetime,
    ) -> dict:
        cached = self.cache.get(user_id, "distribution", query_key,
now)
        if cached is not None:
```

```
            return {
                "value": cached,
                "source": "cache",
            }

        today = now.date()
        if not self.budget_manager.can_spend(user_id, epsilon,
today):
            raise ValueError("privacy budget exceeded")

        params = DPParams(epsilon=epsilon, sensitivity=1.0)
        noisy_counts = privatize_counts_dict(true_counts, params)

        self.budget_manager.spend(user_id, epsilon, today)
        self.cache.set(user_id, "distribution", query_key,
noisy_counts, now)

        return {
            "value": noisy_counts,
            "source": "fresh_dp",
        }
```

十二、第五步：接到现有后端接口里

假设现在已经有 /api/stats/risk_distribution 这样的接口。
原来可能是这样：

```
@app.get("/api/stats/risk_distribution")
def get_risk_distribution():
    true_counts = event_service.query_risk_distribution()
    return jsonify(true_counts)
```

现在应该改成：

```
from datetime import datetime

budget_manager = BudgetManager(daily_limit=3.0)
cache = DPResultCache(ttl_minutes=10)
stats_service = StatsService(budget_manager=budget_manager,
cache=cache)
```

```
@app.get("/api/stats/risk_distribution")
def get_risk_distribution():
    user_id = current_user.id
    start = request.args.get("start")
    end = request.args.get("end")

    true_counts = event_service.query_risk_distribution(start=start,
end=end)

    result = stats_service.get_private_distribution(
        user_id=str(user_id),
        query_key=f"{start}_{end}",
        true_counts=true_counts,
        epsilon=0.8,
        now=datetime.now(),
    )
    return jsonify(result)
```

这样前端拿到的已经是私有化后的分布。

十三、分类统计和按天趋势怎么做

1. 风险等级分布

例如真实数据：

```
{
    "LOW": 7,
    "MEDIUM": 11,
    "HIGH": 4
}
```

直接逐项加噪声即可。

2. 按天趋势

例如真实数据：

```
{
    "2026-04-10": 3,
```

```
    "2026-04-11": 5,
    "2026-04-12": 2,
    "2026-04-13": 6
}
```

一样逐天加噪声即可。
实现上还是调 privatize_counts_dict()。

```
daily_counts = {
    "2026-04-10": 3,
    "2026-04-11": 5,
    "2026-04-12": 2,
    "2026-04-13": 6,
}
noisy_daily_counts = privatize_counts_dict(daily_counts,
DPParams(epsilon=0.8))
```

十四、后处理必须做，但要知道它会引入偏差

NIST 特别提醒，后处理虽然不破坏 DP 保证，但会影响结果偏差和可用性。(NIST 出版物)

建议做这三个后处理：

1. 负值截断为 0

因为事件次数不可能是负数。

2. 四舍五入为整数

因为图表展示的是次数。

3. 低频值做阈值隐藏

例如：

- 如果扰动后仍然小于 5，直接显示 "<5"

这样对小样本更稳。

可以加一个工具函数：

```
def postprocess_count_for_display(noisy_count: int,
low_freq_threshold: int = 5) -> str | int:
    if noisy_count < low_freq_threshold:
        return f"<{low_freq_threshold}"
    return noisy_count
```

十五、建议前端怎么展示

为了避免学生觉得“图表怎么和真实数不完全一样”，前端最好做两件事：

1. 给统计页加一个说明

比如：

为保护个体隐私，统计结果采用隐私增强发布，显示值可能与原始计数存在轻微差异，但不影响总体趋势判断。

2. 不要在同一页同时展示真实值和私有化值

否则这套保护就没意义了。

十六、给一个更完整的可运行示例

下面这段代码把前面几块拼起来了，适合作为学生练手版本。

```
from datetime import datetime
from backend.dp_stats.mechanisms import DPParams, privatize_count,
privatize_counts_dict
from backend.dp_stats.budget_manager import BudgetManager
from backend.dp_stats.cache_service import DPResultCache
from backend.dp_stats.stats_service import StatsService

# 初始化
```

```
budget_manager = BudgetManager(daily_limit=3.0)
cache = DPResultCache(ttl_minutes=10)
stats_service = StatsService(budget_manager=budget_manager,
cache=cache)

# 假设这是从数据库查出来的真实统计
true_total_events = 23
true_risk_distribution = {
    "LOW": 7,
    "MEDIUM": 11,
    "HIGH": 5
}
true_daily_trend = {
    "2026-04-10": 3,
    "2026-04-11": 5,
    "2026-04-12": 2,
    "2026-04-13": 6,
    "2026-04-14": 7
}

user_id = "teacher_001"
now = datetime.now()

# 1) 获取私有化后的总事件数
private_total = stats_service.get_private_total_count(
    user_id=user_id,
    query_key="2026-04-10_to_2026-04-14_total",
    true_count=true_total_events,
    epsilon=0.8,
    now=now,
)

# 2) 获取私有化后的风险分布
private_risk = stats_service.get_private_distribution(
    user_id=user_id,
    query_key="2026-04-10_to_2026-04-14_risk",
    true_counts=true_risk_distribution,
    epsilon=0.8,
    now=now,
)

# 3) 获取私有化后的按天趋势
private_daily = stats_service.get_private_distribution(
    user_id=user_id,
```

```
    query_key="2026-04-10_to_2026-04-14_daily",
    true_counts=true_daily_trend,
    epsilon=0.8,
    now=now,
)

print("private_total =", private_total)
print("private_risk =", private_risk)
print("private_daily =", private_daily)
print("spent_epsilon =", budget_manager.get_spent(user_id,
now.date()))
```

十七、怎么测试这块是不是“基本可用”

这部分也要教给学生。

测试 1：功能正确性

输入真实计数 12，多次调用后，结果应该是：

- 大多在 8 到 16 这类合理区间附近波动
- 不会经常出现离谱负数
- 不会报错

测试 2：缓存是否生效

同一用户、同一图表、同一时间段，在 10 分钟内重复查询，返回结果应相同。

测试 3：预算是否生效

如果一天预算是 3.0，而每次查询用 0.8，那么查 4 次后应该接近上限，第 5 次可能拒绝。

测试 4：低频保护是否生效

真实值为 1、2、3 这些小值时，前端尽量显示 <5，避免小样本泄露。

十八、还可以选一个现成库，但不一定非要上

如果负责同学基础比较弱，手写上面这套就够用了。
如果以后想更规范一点，可以看现成库：

- **OpenDP**：官方文档对 Laplace 机制、查询构造和理论解释比较系统，也有 Python 绑定。(OpenDP)
- **IBM diffprivlib**：是一个比较知名的 Python 差分隐私库，适合做实验和原型。(GitHub)

对现在的建议是：

先把“自己写的轻量计数 DP 模块”做对。
因为这次只需要统计发布层，不需要一上来引入复杂依赖。

十九、一定要避免的 8 个常见坑

1. 把 DP 加在实时告警上

这是错的，会影响功能。

2. 不定义 privacy unit

这样答辩时说不清保护对象。
NIST 明确要求先定义清楚。(NIST 出版物)

3. 只写“们加了噪声”，但不写 ε

这样不算完整方案。
参数必须交代。(NIST 出版物)

4. 不做预算管理

反复查询可能把真实值刷出来。

5. 不做缓存

同一图表每次噪声不同，用户多刷几次就能平均逼近真实值。

6. 对很小的统计值直接原样显示

小样本最容易泄露。

7. 数据库存的就是加噪后的值

不建议。
数据库保留真实事件，发布时再加噪更灵活。

8. 同时展示真实值和私有化值

那就等于没保护。

二十、作品报告里这部分怎么写最稳

可以写成下面这段：

为降低长期聚合统计结果对个体行为模式的反推风险，系统在统计发布层引入轻量差分隐私机制。该机制仅作用于 Dashboard 和统计分析页面中的聚合结果，如日/周事件数、风险等级分布和夜间异常趋势，而不作用于实时检测、即时告警或单条事件详情，从而避免影响系统实时性与处置准确性。
在实现上，系统以被监护对象为隐私单元，对计数类查询采用 Laplace 机制进行私有化处理。具体而言，后端在查询数据库得到真实统计值后，按照设定的隐私预算参数 ε 为结果加入随机噪声，并结合非负截断、低频阈值隐藏和短时缓存机制，兼顾统计可用性与隐私保护。与此同时，系统通过简单的预算管理模块限制单用户在给定时间内的累计查询次数，以降低重复查询导致的隐私泄露风险。
因此，本作品中的差分隐私不是作用于核心识别算法，而是作为统计发布层的隐私增强机制，与前端展示和事件分析模块解耦部署，更适合当前竞赛原型的工程实现条件。

这段写法和 NIST 强调的几个关键点是对齐的：定义隐私单元、明确查询模型、说明参数、说明预算和后处理。(NIST 出版物)

二十一、负责这个模块的同学，最短开发路线

如果要直接安排任务，可以这样分：

第一天

先读懂概念，只做两件事：

- 明白只对统计接口加噪
- 明白 privacy unit 是“一个老人”

第二天

写 mechanisms.py

- 实现 DPParams
- 实现 privatize_count
- 实现 privatize_counts_dict

第三天

写 budget_manager.py

- 每用户每天累计 ε
- 超限报错

第四天

写 cache_service.py

- 同图表 10 分钟缓存

第五天

接到后端接口

- 风险分布图
- 每日趋势图
- 总事件数卡片

第六天

前端展示优化

- <5 低频显示
- 提示“统计结果经过隐私增强处理”

第七天

测试与答辩稿

- 测试预算
- 测试缓存
- 测试低频保护
- 准备解释“为什么只加在统计发布层”

二十二、给一个最直接的落地建议

现在最该做的是这 6 件事：

1. 新建 dp_stats/ 模块。
2. 实现 Laplace + epsilon=0.8 + sensitivity=1 的最小版本。
3. 只接到 3 个接口：总事件数、风险分布、按天趋势。
4. 加预算管理。
5. 加 10 分钟缓存。
6. 小于 5 的值统一显示 <5。

做到这里，这块就已经不是“空谈差分隐私”，而是一个**真实可运行的轻量差分隐私统计发布模块**了。